

Activity 4 - Vibe-code a command parser

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K2/A2

Specification

`parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list | add <sku> <qty> | remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain reading command-line arguments, slice patterns, parsing text into numbers and returning `Result` to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is `samples/01_command_line_args.rs` from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on reading command-line arguments, slice patterns, parsing text into numbers and returning `Result` with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles command-line input and parsing with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: `parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list | add <sku> <qty> | remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Vibe-code a command parser" in `starter/src` so that the supplied tests pass. Generate the parser from the command grammar only; reject any suggestion that adds crates such as `clap`. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_command_line_args.rs

Run: rustc --edition 2021 01_command_line_args.rs && ./01_command_line_args add ABC-0001 5

```
// Sample: reading command-line arguments.
use std::env;
```

```
fn main() {
    let args: Vec<String> = env::args().skip(1).collect();
    if args.is_empty() {
        println!("no arguments: try ./01_command_line_args add ABC-0001 5");
        return;
    }
    for (index, arg) in args.iter().enumerate() {
        println!("argument {index}: {arg}");
    }
}
```

Expected output:

```
argument 0: add
argument 1: ABC-0001
argument 2: 5
```

02_parse_with_result.rs

Run: rustc --edition 2021 02_parse_with_result.rs && ./02_parse_with_result

```
// Sample: turning text into a number safely with Result and ?.
```

```
fn parse_qty(text: &str) -> Result<u32, String> {
    let qty: u32 = text
        .parse()
        .map_err(|_| format!("not a whole number: {text}"))?;
    if qty == 0 {
        return Err("quantity must be at least 1".to_string());
    }
    Ok(qty)
}

fn main() {
    for input in ["5", "0", "five", "-3"] {
        match parse_qty(input) {
            Ok(qty) => println!("{input}:>5} -> Ok({qty})",
            Err(message) => println!("{input}:>5} -> Err({message})",
        }
    }
}
```

Expected output:

```
5 -> Ok(5)
0 -> Err(quantity must be at least 1)
five -> Err(not a whole number: five)
-3 -> Err(not a whole number: -3)
```

Learn more

Rust Book: Programming a Guessing Game:

<https://doc.rust-lang.org/book/ch02-00-guessing-game-tutorial.html>

Rust Book: An I/O Project (command-line program):

<https://doc.rust-lang.org/book/ch12-00-an-io-project.html>

W3Schools: Rust Functions: https://www.w3schools.com/rust/rust_functions.php

Programiz: Rust Function: <https://www.programiz.com/rust/function>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php